

OPEN SOURCE, PYTHON, MIT

SGR Agent Core

Как и зачем начать им пользоваться

Павел Рыков - Pavel Zloi

ЗНАКОМСТВО

Всем привет, я Павел Рыков

- ◆ Известен как **Павел Злой**, руководитель импортозамещения ML в Росгосстрах
- ◆ ML и LLM, агентные системы, self-host и инференс, open-source и телеграм-канал
- ◆ В SGR Agent Core отвечаю за **разработку API** - HTTP-сервер и ACP
- ◆ Проект развивает сообщество neuraldeer и vamlabAI при поддержке **red_mad_robot**



@EVILFREELANCER

t.me/evilfreelancer

канал Pavel Zloi - наведите камеру

ПРОГРАММА

О чём поговорим

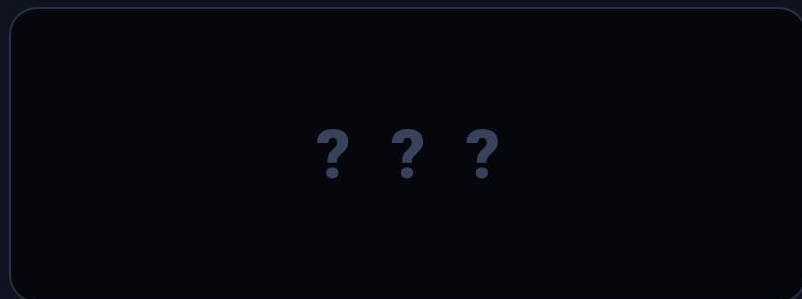
- ◆ Что такое SGR и зачем он нужен
- ◆ Архитектура - API и ACP
- ◆ Системные тулы и встроенные агенты
- ◆ YAML-конфигурация и реестры
- ◆ Как собрать свой тул и агента
- ◆ Как запустить и куда подключить
- ◆ Deep Research и файловый агент
- ◆ Метрики, тесты и Langfuse

Теория, компоненты и короткие примеры кода

ИДЕЯ

Что такое Schema-Guided Reasoning

Классический агент - чёрный ящик



вход - ??? - выход

VS

SGR-агент - прозрачный

Рассуждение

JSON-схема

Планирование

JSON-схема

Действие

JSON-схема

Ответ

JSON-схема

Вместо непрозрачного ящика - явные фазы, каждая описана схемой и проверяется

СХЕМА В КОДЕ

Та самая структура - pydantic-модель

```
class ReasoningTool(SystemBaseTool):  
    reasoning_steps: list[str]    # ход мысли, 2-3 шага  
    current_situation: str        # где мы сейчас  
    plan_status: str             # статус плана  
    enough_data: bool            # хватает данных  
    remaining_steps: list[str]   # что осталось  
    task_completed: bool         # задача закрыта
```

После конвертации в JSON схема уходит в модель, каждое поле - обязательный шаг

АРХИТЕКТУРА

Один агент, три способа запуска

API

HTTP-сервер

OpenAI-совместимый эндпоинт, стриминг по SSE, stateless и stateful. Open WebUI, LibreChat, любой OpenAI-клиент

ACP

Редакторы по stdio

Agent Client Protocol, агент как подпроцесс по JSON-RPC. Obsidian, Zed, VS Code, Claude Desktop

CLI

Терминал

Интерактивный шелл и одиночные запросы прямо из командной строки

Логика агента одна, меняется только способ запуска и один и тот же конфиг

Agent Client Protocol

Что это

- ◆ Протокол для локальных агентов поверх stdio
- ◆ Агент запускается как подпроцесс
- ◆ Общение через JSON-RPC
- ◆ Поддержка tools, resources, prompts

Хосты

- ◆ Obsidian с плагином Agent Client
- ◆ Zed - нативная панель агента
- ◆ VS Code, JetBrains и другие IDE
- ◆ Claude Desktop

Запускается командой `sgracr` с тем же конфигом, что и HTTP-сервер

API Mode - REST-сервер

Жизненный цикл

- ◆ Создание - загрузка конфига, инициализация тулов
- ◆ Выполнение - обработка запроса, SGR-пайплайн
- ◆ Очистка - освобождение ресурсов

Режимы

- ◆ Stateless - каждый запрос независимый
- ◆ Stateful - контекст диалога по agent_id
- ◆ Стриминг по SSE, отмена выполнения

FastAPI-сервер с хранилищем агентов, совместим с любым OpenAI-клиентом

КОМПОНЕНТЫ - ТУЛЫ

Системные тулы

- ◆ `reasoning_tool` анализ и следующий шаг
- ◆ `generate_plan_tool` генерация плана
- ◆ `adapt_plan_tool` адаптация плана
- ◆ `clarification_tool` запрос уточнения
- ◆ `final_answer_tool` финализация ответа
- ◆ `answer_tool` промежуточный ответ
- ◆ `web_search_tool` Tavily, Brave, Perplexity
- ◆ `extract_page_content_tool` контент из URL
- ◆ `create_report_tool` отчёт с цитатами
- ◆ `run_command_tool` команды и код в песочнице

Встроенные агенты

- ◆ `sgr_agent` максимально строгий и управляемый SGR
- ◆ `sgr_tool_calling_agent` SGR плюс нативный tool calling, больше совместимости
- ◆ `tool_calling_agent` тулколлинг без SGR
- ◆ `dialog_agent` диалог с промежуточными результатами
- ◆ `iron_agent` упрощённый, без tool calling

Любой можно расширить своим наследником, механизм ядра общий

КОНФИГУРАЦИЯ

Иерархия настроек - конфиг вместо кода

Дефолты фреймворка

->

Переменные окружения

->

config.yaml

->

Параметры CLI

Каждый следующий слой переопределяет предыдущий. Секции конфига:

- ◆ `llm` настройки LLM
- ◆ `execution` лимиты и директории
- ◆ `tools` регистрация тулов
- ◆ `mcp` MCP-серверы
- ◆ `agents` определение агентов
- ◆ `acp` настройки ACP

Регистрируем тулы в config.yaml

```
tools:
  # простая регистрация
  reasoning_tool: {}
  final_answer_tool: {}

  # с конфигурацией
  web_search_tool:
    engine: "tavily"          # или brave, perplexity
    api_key: "${TAVILY_API_KEY}"
    max_results: 10

  # кастомный тул из модуля
  read_file_tool:
    base_class: "tools.ReadFileTool"
```

Подключаем внешние MCP-серверы

```
mcp:
  mcpServers:
    # удалённый сервер по URL
    deepwiki:
      url: "https://mcp.deepwiki.com/mcp"

    # локальный stdio-сервер
    filesystem:
      command: "npx"
      args: ["-y", "@modelcontextprotocol/server-filesystem"]
```

Агент автоматически получает тулы от подключённых MCP-серверов

Определяем агента в config.yaml

```
agents:
  sgr_tool_calling_agent:
    base_class: "agents.ResearchSGRToolCallingAgent"
    llm:
      model: "gpt-4.1-mini"
      temperature: 0.4
    tools:
      - "reasoning_tool"          # обязателен для SGR
      - "web_search_tool"
      - "create_report_tool"
      - "final_answer_tool"
```

СВЯЗЬ КОМПОНЕНТОВ

Агент, тулы и MCP

Агент - SGR Pipeline / Tool Calling

Reasoning

->

Planning

->

Acting



Tools Layer - зарегистрированные тулы

web_search_tool

extract_page_content_tool

create_report_tool

run_command_tool



MCP Servers - внешние

deepwiki

filesystem

git

...

СВОЙ ИНСТРУМЕНТ

Тул это схема плюс один метод

```
from sgr_agent_core.base_tool import BaseTool
from pydantic import Field

class FindTool(BaseTool):
    tool_name = "find_tool"
    reasoning: str = Field(description="зачем ищем файлы")
    pattern: str = Field(description="glob-паттерн имени")

    async def __call__(self, context, config) -> str:
        return find_files(self.pattern) # текст для модели
```

Поля Field это одновременно аргументы и подсказка модели,
как их заполнить

СВОЙ АГЕНТ

Своя логика - наследование

```
from sgr_agent_core.agents import SGRToolCallingAgent

class SGRFileAgent(SGRToolCallingAgent):
    name = "sgr_file_agent"
    toolkit = [ListDirTool, ReadFileTool,
               FindTool, GrepTool]

    # working_directory ограничивает песочницу
    # при желании переопределяем шаги цикла
```

Наследник регистрируется в ядре автоматически и доступен по имени в конфиге

КАК НАЧАТЬ

Установка и запуск

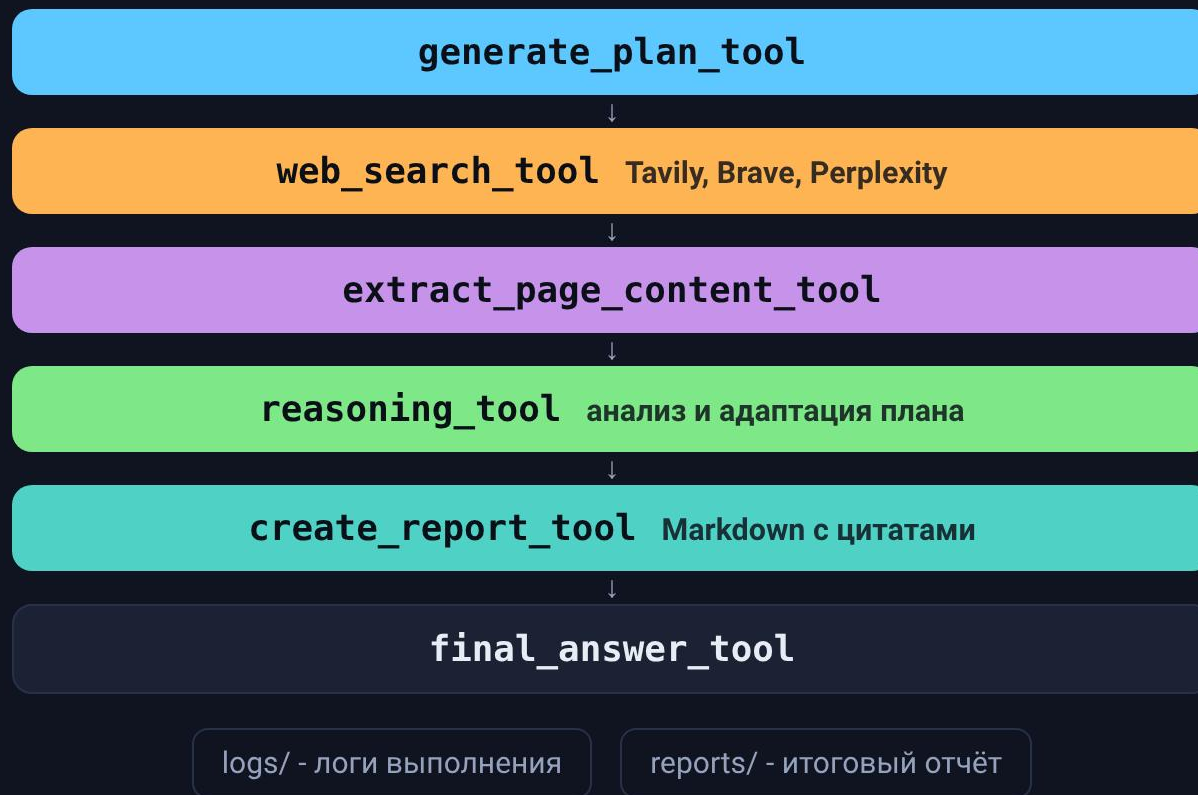
```
# поставить пакет
pip install sgr-agent-core

# скопировать и заполнить ключи в config.yaml, затем запустить
sgr      -c config.yaml      # HTTP-сервер, OpenAI-совместимый API
sgrsh    -c config.yaml      # интерактивный CLI
sgracp   -c config.yaml      # агент по протоколу ACP через stdio
```

Дальше обращаемся как к обычному OpenAI API, где model это имя агента

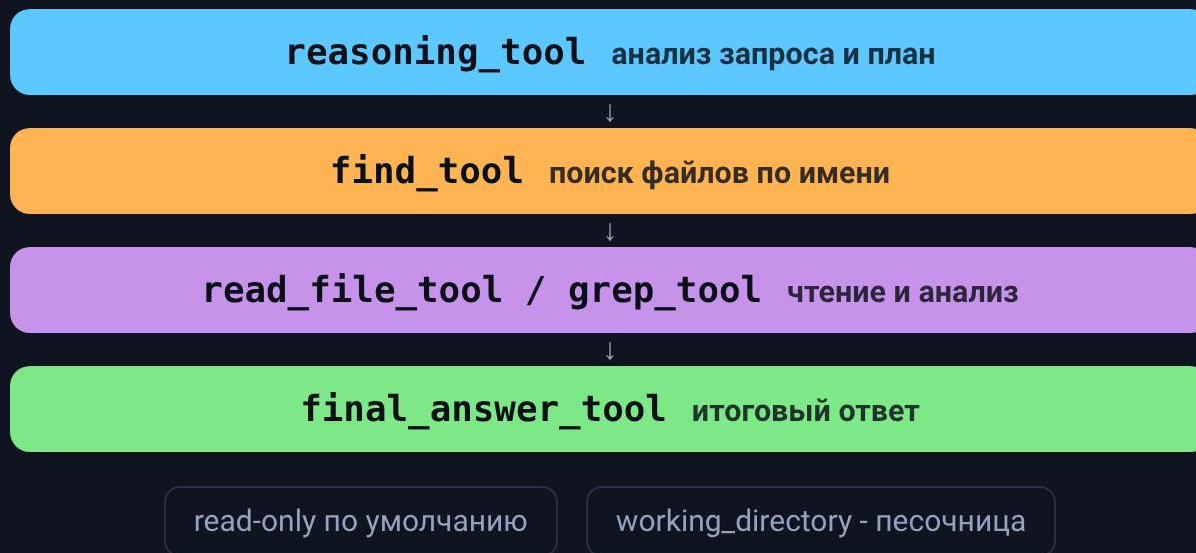
ПРИМЕР - DEEP RESEARCH

Готовый агент исследования



ПРИМЕР - ФАЙЛОВЫЙ АГЕНТ

Агент по локальным файлам



КУДА ПОДКЛЮЧАЕТСЯ

Готовые клиенты без своего UI

Через OpenAI API

- ◆ Open WebUI - локальный веб-интерфейс
- ◆ LibreChat - мульти-модельный чат
- ◆ Continue и любой клиент с OPENAI_BASE_URL

Через ACP

- ◆ Obsidian - плагин Agent Client
- ◆ Zed - нативная панель агента
- ◆ VS Code и JetBrains через расширения ACP

Берём готовый клиент и указываем адрес агента, свой фронт писать не нужно

НАБЛЮДАЕМОСТЬ

Метрики, тесты и Langfuse

- ◆ Каждый шаг агента виден в логах - прозрачный пайплайн, а не чёрный ящик
- ◆ Тулы тестируются как обычные классы - unit-тест на вход и выход
- ◆ Langfuse подключается блоком в конфиге - трейсы, задержки, стоимость
- ◆ Метрики - полнота ответа, число лишних вызовов, регрессия при смене модели

Поверх видимых шагов можно строить метрики и
сравнивать прогоны

ИТОГ

Зачем начинать пользоваться

- ◆ **Предсказуемость** рассуждение по схеме, без пропущенных шагов
- ◆ **Переносимость** любой OpenAI-совместимый провайдер и локальные модели
- ◆ **Расширяемость** свои тулы, схемы и агенты через конфиг и наследование
- ◆ **Экосистема** готовые клиенты через OpenAI API и ACP, MCP, Langfuse

github.com/vamplabAI/sgr-agent-core - проект команды neuraldeep и
vamplabAI при поддержке red_mad_robot, концепция SGR - LLM Under
the Hood

СПАСИБО ЗА ВНИМАНИЕ

Вопросы?

Пишите в канал, там разборы про агентов, SGR и self-host без маркетинга.



t.me/evilfreelancer

Павел Рыков - Pavel Zloi

